바이브 코딩 7장 · 배포와 데이터

Supabase 사용법

정적 페이지에 로그인과 데이터 저장을 붙이기

한신대학교 AI·SW계열 · 웹프로그래밍

학습사이트 hs-web.dreamitbiz.com

학습 목표

① Supabase가 무엇을 대신해 주는지 안다

② 표를 만들고 웹페이지에서 읽고 쓸 수 있다

③ anon 키가 왜 공개돼도 되는지, 그 전제가 무엇인지 안다

바이브 코딩 7장

핵심 키워드

Supabase PostgreSQL RLS anon 키 인증

학습사이트의 본문에서 이 낱말들을 찾아 읽으면 이 시간의 뼈대가 잡힙니다

바이브 코딩 7장 · 1

무엇을 대신해 주나

지금까지 만든 페이지는 새로고침하면 다 사라집니다.

무엇을 대신해 주나

1. 저장할 곳이 없기 때문입니다.
2. 원래는 서버를 만들고 데이터베이스를 깔아야 하는데, Supabase 는 그것을 대신해 줍니다.

무엇을 대신해 주나

원래 필요한 것	Supabase가 주는 것
데이터베이스 서버	Database — 표에 데이터를 저장
로그인 처리 서버	Auth — 구글·카카오 로그인을 붙임
파일 저장 서버	Storage — 이미지·파일 업로드
API 서버 코드	표를 만들면 읽고 쓰는 주소가 자동으로 생김

무엇을 대신해 주나

1. 이 사이트도 Supabase를 씁니다.
2. 여러분의 진도·출결·시험 기록이 거기에 저장됩니다.
3. 로그인 버튼을 누를 때 일어나는 일이 바로 이 장의 내용입니다.

바이브 코딩 7장 · 2

① 프로젝트 만들기

① 프로젝트 만들기

1. supabase.com 에서 가입합니다. GitHub 계정으로 가입하면 간단합니다(6장).
2. **New project**
   를 누릅니다.
3. 이름을 정하고, 데이터베이스 비밀번호를 만듭니다. 이건 어디에도 적어 두지 말고 비밀번호 관리자에 저장하세요.
4. 지역(Region)은 Northeast Asia (Seoul) 처럼 가까운 곳을 고릅니다. 멀면 느려집니다.
5. 만들어지는 데 1~2분 걸립니다.

① 프로젝트 만들기

1. 무료로 쓸 수 있는 범위와 조건은 자주 바뀌므로 여기 적지 않습니다.
2. 공식 요금 페이지에서 지금 조건을 확인하세요.
3. 수업 과제 규모에서는 대개 무료로 충분합니다.

② 두 개의 키 — 이 장에서 가장 중요합니다

프로젝트를 만들면 Settings → API 에 값 세 개가 있습니다.

② 두 개의 키 — 이 장에서 가장 중요합니다

이름	무엇	코드에 넣어도 되나
Project URL	내 프로젝트 주소	넣어도 됩니다
anon (public) key	"익명 방문자" 자격으로 접근하는 키	넣어도 됩니다 — 단, 아래 전제가 있습니다
service_role key	모든 잠금을 무시하는 관리자 키	절대 안 됩니다

이 중 두 키의 차이를 반드시 알아야 합니다.

service_role 키는 브라우저 코드에 절대 넣지 않습니다

정리

1. 이름 그대로 모든 보안 규칙을 통과합니다. 남의 데이터도 전부 읽고 지울 수 있습니다.
2. 5장에서 배운 대로, 브라우저로 내려간 코드는 누구나 볼 수 있습니다.
3. 이 키가 필요한 작업은 서버에서만 합니다. 우리 과제 범위에서는 쓸 일이 없습니다.
4. 화면에서 복사할 때 두 키가 나란히 있으니 헷갈리지 않도록 조심하세요.

anon 키는 왜 공개돼도 되나

1. 5장에서는 "키를 코드에 넣지 마라"고 했는데 여기서는 넣으라고 합니다.
2. 모순이 아닙니다.
3. anon 키는 "누구세요"를 말할 뿐, "무엇을 할 수 있는지"는 정하지 않기 때문입니다.

anon 키는 왜 공개돼도 되나

	AI API 키 (5장)	Supabase anon 키
가진 사람은	나로서 돈을 쓸 수 있다	익명 방문자로 문을 두드릴 수 있다
할 수 있는 일	내 한도까지 전부	내가 허용한 것만
누가 정하나	없음 (키가 곧 권한)	내가 RLS로 정한다
공개되면	요금 폭탄	설계대로면 괜찮다

단, 전제가 있습니다 — RLS를 켰을 때만

정리

1. **RLS(Row Level Security)**
   는 "어떤 행을 누가 볼 수 있는가"를 정하는 잠금입니다.
2. **RLS를 끄면 anon 키만으로 그 표의 모든 데이터를 읽고 지울 수 있습니다.**
   키가 코드에 있으니 누구나요.
3. **표를 만들면 Supabase가 "RLS가 꺼져 있다"고 경고를 띄웁니다. 그 경고를 무시하지 마세요.**
4. **가장 흔한 사고: 실습하느라 RLS를 껐다가 켜는 걸 잊고 배포하는 것.**

바이브 코딩 7장 · 4

③ 표 만들기

왼쪽 메뉴의 SQL Editor 에서 아래를 붙여넣고 실행(Run)합니다.

방명록 표 만들기 — 이름·메시지·작성시각 (1/2)

```
-- 표 만들기
create table if not exists guestbook (
  id         bigint generated always as identity primary key,
  nickname   text not null,
  message    text not null,
  created_at timestamptz not null default now()
);

-- 잠금 켜기 — 이 줄을 빠뜨리면 누구나 전부 지울 수 있습니다
alter table guestbook enable row level security;
```

> **화면에서 클릭으로 만들 수도 있지만, SQL로 해 두면 나중에 그대로 다시 쓸 수 있습니다.**

방명록 표 만들기 — 이름·메시지·작성시각 (2/2)

```
19  -- 규칙 1: 누구나 읽을 수 있다 (방명록이니까)
20  create policy "누구나 읽기" on guestbook
21    for select to anon using (true);
22
23  -- 규칙 2: 누구나 새 글을 쓸 수 있다
24  create policy "누구나 쓰기" on guestbook
25    for insert to anon with check (true);
26
27  -- 규칙 3: 수정 · 삭제 규칙은 만들지 않는다
28  --         → 정책이 없으면 막힙니다. 남의 글을 지울 수 없게 됩니다.
```

RLS의 기본은 "전부 막힘"입니다

정리

1. RLS를 켜면 정책을 만든 동작만 열립니다. 안 만든 것은 자동으로 막힙니다.
2. 위 예제에서 update·delete 정책을 안 만든 것은 일부러입니다. 방명록 글을 남이 고칠 수 없어야 하니까요.
3. "기록을 남기는 표에는 수정·삭제를 열지 않는다" — 실무에서도 쓰는 원칙입니다.
4. 반대로 using (true) 는 "누구든 통과"라는 뜻입니다. 개인정보가 든 표에는 절대 쓰지 마세요.

바이브 코딩 7장 · 5

④ 웹페이지에서 읽고 쓰기

guestbook.html — 한 파일로 되는 방명록 (1/4)

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>방명록</title>
</head>
<body>
  <h1>방명록</h1>

  <form id="form">
    <input id="nickname" placeholder="이름" required>
    <input id="message" placeholder="한마디" required>
    <button type="submit">남기기</button>
  </form>

  <ul id="list"></ul>
```

guestbook.html — 한 파일로 되는 방명록 (2/4)

```
19    <script type="module">
20      import { createClient } from 'https://cdn.jsdelivr.net/npm/@supabase/supabase-js/+esm';
21
22      // Settings → API 에서 복사합니다 (anon 키만! service_role 아님)
23      const db = createClient(
24        'https://<내프로젝트>.supabase.co',
25        '<여기에_anon_키>'
26      );
27
28      const list = document.getElementById('list');
29
30      // 읽기 — 최신 글부터 20개
31      async function load() {
32        const { data, error } = await db
33          .from('guestbook')
```

guestbook.html — 한 파일로 되는 방명록 (3/4)

```
      .select('*')
      .order('created_at', { ascending: false })
      .limit(20);

    if (error) { console.error(error); return; }

    list.innerHTML = '';
    for (const row of data) {
      const li = document.createElement('li');
      li.textContent = row.nickname + ': ' + row.message;
      list.appendChild(li);   // innerHTML 이 아니라 textContent 를 씁니다
    }
  }

  // 쓰기
  document.getElementById('form').addEventListener('submit', async (e) => {
```

guestbook.html — 한 파일로 되는 방명록 (4/4)

```
      e.preventDefault();
      const nickname = document.getElementById('nickname').value;
      const message  = document.getElementById('message').value;

      const { error } = await db.from('guestbook').insert({ nickname, message });
      if (error) { alert('저장 실패: ' + error.message); return; }

      e.target.reset();
      load();
    });

    load();
  </script>
</body>
</html>
```

남이 쓴 글을 화면에 넣을 때는 textContent

정리

1. innerHTML 로 넣으면 방문자가 적은 HTML이 그대로 실행됩니다. 남의 스크립트가 내 페이지에서 도는 것입니다.
2. textContent 는 글자로만 넣기 때문에 안전합니다.
3. 이것을 XSS 라고 부릅니다. 13주차(이벤트 처리)에서 다시 나옵니다.
4. AI가 innerHTML 을 쓴 코드를 줄 때가 많습니다. 받은 코드에서 이걸 찾아 고치는 것이 좋은 연습입니다.

⑤ 로그인 붙이기 (선택)

Authentication → Providers 에서 구글·카카오 등을 켜면 로그인을 붙일 수 있습니다.

로그인·로그아웃·현재 사용자

```
// 구글로 로그인
await db.auth.signInWithOAuth({
  provider: 'google',
  options: { redirectTo: window.location.origin }
});

// 지금 로그인한 사람은 누구인가
const { data: { user } } = await db.auth.getUser();
console.log(user?.email);

// 로그아웃
await db.auth.signOut();
```

> 각 서비스에서 발급받은 값을 넣어야 하므로 준비할 것이 조금 있습니다.

Redirect URL을 등록하지 않으면 조용히 실패합니다

정리

1. **Authentication → URL Configuration → Redirect URLs**
   에 내 페이지 주소를 등록해야 합니다.
2. **등록하지 않으면 로그인은 되는데 엉뚱한 곳으로 튕깁니다. 오류 메시지가 안 뜨니 원인을 찾기 어렵습니다.**
3. **GitHub Pages 주소(https://내이름.github.io/...)를 그대로 넣으세요.**
4. **주소가 바뀌면 다시 등록해야 합니다.**

바이브 코딩 7장 · 7

내 데이터가 안전한지 스스로 확인하기

만들고 나서 꼭 직접 확인하세요.

내 데이터가 안전한지 스스로 확인하기

1. **"설정했으니 됐겠지"가 가장 위험합니다.**
2. **Table Editor에서 각 표에 RLS 표시가 켜져 있는지 봅니다.**
3. **로그아웃한 상태**
   로(또는 시크릿 창에서) 내 페이지를 열어 봅니다. 보이면 안 되는 것이 보이나요?
4. **개인정보가 든 표에 using (true) 정책이 있지는 않은지 봅니다.**
5. **service_role 키가 코드 어디에도 없는지 검색해 봅니다.**

내 데이터가 안전한지 스스로 확인하기

1. 4번은 실제로 이렇게 확인합니다 — VS Code에서 프로젝트 폴더 전체를 service_role 로 검색해 결과가 0건이어야 합니다.
2. 눈으로 훑지 말고 검색하세요.

바이브 코딩 7장 · 8

한 장으로 정리

한 장으로 정리 (1/2)

1. **Supabase는 서버 없이 저장·로그인·파일을 붙여 준다**
2. **anon 키는 공개돼도 된다**
   단 RLS가 켜져 있을 때만
3. **service_role 키는 브라우저에 절대 넣지 않는다**
4. **RLS는 기본이 전부 막힘 — 정책을 만든 동작만 열린다**

한 장으로 정리 (2/2)

1. 기록성 표에는 수정·삭제 정책을 만들지 않는다
2. 남이 쓴 글은 textContent 로 넣는다
3. 로그인은 Redirect URL 등록을 잊지 않는다

이만큼 되면 이 장은 끝입니다

- [ ] Supabase 프로젝트를 만들었다
- [ ] anon 키와 service_role 키의 차이를 설명할 수 있다
- [ ] RLS를 켜고 정책을 만들어 표를 하나 완성했다
- [ ] 웹페이지에서 데이터를 읽고 쓰는 것을 확인했다
- [ ] service_role 키가 내 코드에 없는지 검색으로 확인했다
- [ ] 로그아웃 상태에서 무엇이 보이는지 직접 확인했다

바이브 코딩 7장 마무리

7장 마무리

Supabase 사용법 — 예상 45분 · 읽음 표시는 학습사이트에서

- ✔ 학습사이트의 「이 장을 다 읽었습니다」 단추로 읽음 표시
- ✔ 회차마다 바뀌는 값(가격·한도·버전)은 공식 페이지에서 확인
- ✔ 막히면 「질문있어요」 — 30분 넘게 혼자 막혀 있지 않기